function as inputs, sends a prompt to the AI model, and then validates the response against the provided schema. This ensures that we receive type-safe, structured data that conforms to our expected format.

Using this `generateObject` function along with our product schema, we'll modify the AI server action to request a JSON output. This modification will allow us to generate structured product information that can be easily parsed and displayed in our application. The JSON format will enable us to handle complex data structures more efficiently, making it easier to present product details in a user-friendly manner, such as in tables or cards.

CHANGES TO SERVER ACTIONS

To refactor our server actions to use these tools, we only need to export the `generateProductList` instead of the `continueConversation` in the list of actions since we will be changing the response format of the messages.

Listing 4.11 Including the `generateProductList` action

```
export const AI = createAI({
  actions: {
    generateProductList,
  },
  initialAIState: [],
  initialUIState: [],
});
```

Exports the generateProductList action here instead of the continueConversation

With this change, we are now exporting the new server action to generate a structured response. We now simply need to update the UI to render the new message format.

> **NOTE** When working with `createAI`, it's important to include all your server actions within the `createAI` instance, rather than importing them directly in your components. The main reason is that `createAI` creates a context that makes these actions easily accessible throughout your component tree, even in deeply nested client-side components, via the `useActions` hook. Additionally, when used with TypeScript, `createAI` provides type checking for your actions, reducing the likelihood of runtime errors.

CHANGES TO THE FRONTEND

Regarding the changes to the frontend, we need to modify two components:

- `ChatList`—This component now renders not only the list of user prompt history but the list of products from the assistance response result. This means that it needs to render the `ProductListSchema` type in a table format. I do not show exactly how this is implemented, but you can review the code of this component in the provided code sources.
- page.js—The main chat page component needs to update the form submit handler to use the new `generateProductList` server action instead of the `continueConversation` action.